

Wrap-up report

1. 프로젝트 개요

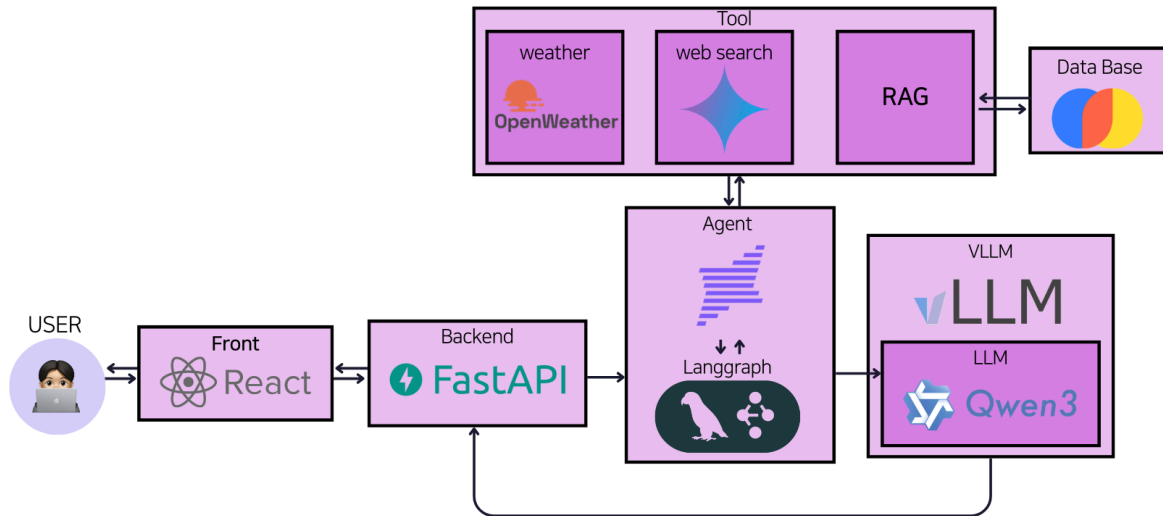
1.1 프로젝트 개요

딱딱한 비서형 LLM이 아니라, 카카오톡 대화처럼 자연스럽게 여행 계획을 함께 완성해주는 친구형 여행 에이전트 구현을 프로젝트 주제로 선정하였다. 병렬적인 Tool 실행과 (검색-검증-보완) 같은 순환형 워크플로를 안정적으로 구성하기 위해 LangGraph를 도입하였다. 또한 친근한 말투와 대화 흐름을 위해 모델을 AI-Hub의 한국어 SNS 멀티턴 대화 데이터로 SFT 및 DPO 방식으로 QLoRA 학습했으며, 이를 에이전트의 대화 모델로 통합하여 챗봇 시스템을 구현하였다.

1.2 개발 환경

Programming Language	Python 3.10.13
Experiment Tracking	Weights & Biases
Version Control	GitHub
Collaboration Tool	Slack
H/W & OS	NVIDIA Tesla V100 (32GB), Ubuntu 20.04 LTS

1.3 프로젝트 구조 및 데이터셋 구조도



2. 프로젝트 팀 구성 및 역할

김영현

- RAG 구축, LangGraph 구현

윤준상

- EDA, 프론트엔드 구현, Agent Tool 설계

장세현

- SFT 데이터 전처리, LangGraph 구현, Agent Tool 설계

주현민

- SFT데이터 수집/EDA , 페르소나 학습, 발표자료 제작 및 발표

한지석

- DPO 학습 데이터 구축, DPO 학습, RAG 평가 파이프라인 설계, 모델 서빙

3. 프로젝트 수행 절차 및 방법

Week/Day	Mon	Tue	Wed	Thur	Fri	Sat	Sun
Jan Week2	프로젝트 주제 선정						
			자료 조사				
			데이터 수집				
Jan Week3	데이터 수집				프론트 구현		
Jan Week4			페르소나 학습 실험				
	프론트 구현						
			RAG 설계 및 실험				
Jan Week5	LangGraph 설계 및 실험						
	페르소나 학습 실험						
Feb Week1	React 구현						
	모델 성능 개선						
			프론트 실험			리포트 작성 및 발표 준비	

4. 프로젝트 수행 결과

4.1 데이터 분석 (EDA)

4.1.1 데이터셋 개요

챗봇의 목적(AI-Hub 한국어 SNS 말투 기반의 친근한 여행 도우미)에 맞추어 **AI Hub 한국어 SNS 멀티턴 대화 데이터셋**을 사용하였다. 본 프로젝트에서는 실제 서비스의 사용자-챗봇 상호작용 형태에 맞추기 위해 여러 주제 중 여행 주제, **발화자 수가 2명인 대화만 선별**하여 최종 학습 데이터로 구성하였다.

4.1.2 데이터 품질 문제 및 개선

초기에는 싱글턴 형태(단일 발화-응답 쌍)로 학습 데이터를 구성했으나, 생성 답변이 지나치게 짧고 문맥을 반영하지 못하는 문제가 발생했다. 이는 데이터 구성이 단순하여 모델이 표면 패턴에 과적합했기 때문으로 분석되었다.

이에 따라 데이터 전처리를 재수행하여 '여행' 주제 대화 중에서도 '**정보 요청 → 정보 제공**' 구간을 **중심으로 추출**하고, 답변이 나오기 위해 필요한 앞 문맥까지 함께 포함하는 방식으로 학습 데이터를 재구성하였다. 이를 통해 모델이 친근한 말투뿐 아니라 적절한 대화 전략(정보 제공 시점, 반문 시점, 계획 확정 방식)을 함께 학습할 수 있도록 설계하였다.

4.2 Model Training

여행 챗봇이 SNS 특유의 친근한 말투(Persona)를 일관되게 유지하면서도 정보 제공 품질을 확보하기 위해 2단계 학습(SFT → DPO)을 적용하였다. SFT로 기본 대화 구조를 학습하고, DPO로 미세한 말투·성향을 선호 기반으로 정렬하였다.

4.2.1 SFT (Supervised Fine-Tuning)

- QLoRA(4-bit quantization) 적용
- 사용자 질문(Instruction) 토큰은 loss 계산에서 제외하고, **모델의 응답(Response) 토큰에 대해서만 loss를 계산**하여 페르소나 학습 효율을 극대화
- 데이터 규모(약 1.3만 대화) 및 과적합 가능성을 고려하여 **1 epoch** 설정
- 추가로 QLoRA 설정은 학습 안정성을 위해 **보수적으로 조정**

4.2.2 DPO (Direct Preference Optimization)

SFT만으로는 말투의 미세한 차이(불필요 반문, 과도한 장황함, 정보 제공 부족 등)를 안정적으로 제어하기 어렵기 때문에, 별도의 Reward Model이나 강화학습 절차 없이도 선호 정렬이 가능한 **DPO**를 도입했다.

“친근함은 유지하되 SNS 대화 특성에 맞게 간결하고 핵심적인 정보 제공 중심으로 답하는 스타일”을 강화하고, 반대로 “과도하게 늘어지는 응답”을 약화시키는 방향으로 선호를 정렬을 목표로 데이터셋을 구축했다.

- 데이터셋 구축
 - Chosen: SFT 데이터(GT 응답)
 - Rejected: LLM(Solar-Pro3)을 이용해 의도적으로 생성한 “친절하지만 장황한 성격을 띄는” 답변
 - 데이터 분할: Train/Eval/Test = 8:1:1, 총 10,000쌍

4. 3 RAG

4.3.1 데이터셋

사용자에게 신뢰성 있는 정보를 제공하기 위해 카카오맵 214개의 데이터와 '비жит부산' 홈페이지의 가이드북 851개의 데이터를 사용하여 총 1,065개의 데이터를 활용하였다.

1. 카카오맵

- 리뷰는 상대적으로 많지 않으나 비판적 의견이 반영되어 있어 현실적인 정보 확보를 위해 Selenium 와 BeautifulSoup를 이용하여 카카오맵 리뷰 데이터를 동적 크롤링

2. '비жит부산' 홈페이지 가이드북

- '비짓부산' 홈페이지의 《블루리본 2025》, 《택슬랭 2025》 등 가이드북 16개
- PyMuPDF와 LLM을 이용해 데이터 정제 및 청크 구성

4.3.2 구축

임베딩 모델은 API 기반 모델을 활용하였다. 그중에서도 한국어 성능이 우수하고 문장 특화 도메인인 solar-embedding-1-large 모델을 선정하였다.

벡터 데이터 베이스는 챗봇 구성에 사용한 LangGraph와의 높은 호환성, 메타데이터 기반 필터링 기능의 편의성을 고려하여 Chroma db를 선정하였다.

검색 성능 향상을 위해 dense retriever와 sparse retriever를 결합한 하이브리드 retriever를 사용하였다. dense retriever는 코사인 유사도 기반 방식으로, sparse retriever는 BM25 기반 방식으로 각각 상위 20개씩 수행한 후, 검색 결과를 통합하여 최종 상위 5개의 문서를 선정하도록 설계하였다.

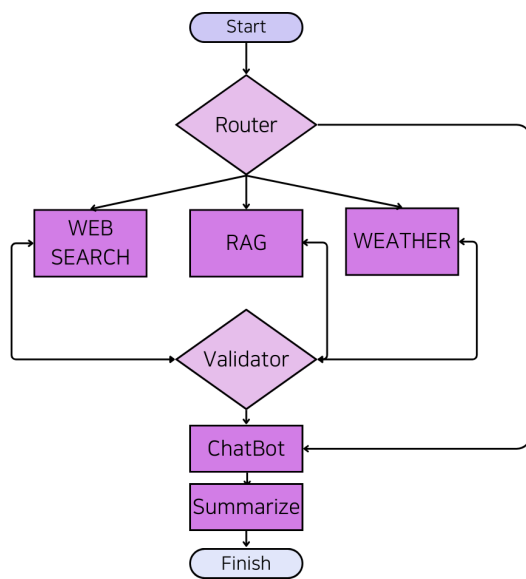
4.3.3 평가

정답(Ground Truth)이 없는 RAG의 성능을 객관적으로 평가하기 위해서는 고품질의 테스트 데이터셋이 필수적이다. 이를 위해 본 프로젝트에서는 Ragas의 지식 그래프 기반 접근법을 통해 데이터 생성 및 평가 파이프라인을 구축했다.

Context Recall과 Precision을 지표로 평가한 결과, **Hybrid 검색(Top-k 20 → 5)은 Recall 0.96**으로 정보 확보에 우수했으며, **Dense 검색(k=5)은 Precision 0.91**로 답변의 정밀도 측면에서 가장 높은 성능을 보였다.

4.4 langgraph

4.4.1 구조



- 단발성 체인이 아닌 State 를 유지하는 에이전트 구조를 설계
- 사용자의 인풋이 들어오면, router 에서 질문을 세분화하여 각 agent 에 나누어진 질문을 할당
- 각 agent 는 질문에 답을 하고 validator 에 state 를 전달 → validator 는 답변이 불충분한 경우 재검색을 수행하도록 하여 정확도를 향상
- router 와 validator 에서 사용된 모델은 gpt-4o 를 사용하여 정확한 쿼리와 속도 향상
- summarize 에서는 이전 대화 맥락을 요약하여 Token Context 관리

4.4.2 Tool

단발성 체인이 아니라 '상태(state)를 유지하는 에이전트 워크플로'를 설계했다.

사용자 입력이 들어오면 Router가 질문을 분석해 적절한 처리 경로(Web Search / RAG / Weather)로 라우팅한다. 각 Agent는 선택된 도구를 실행해 중간 결과를 생성하고, 이를 Validator에 전달한다.

Validator는 결과의 충분성(질문 의도 충족 여부, 근거/정보 누락 여부)을 점검하고, 필요 시 재검색/재조회(루프)를 수행하도록 하여 답변의 신뢰성과 완성도를 높인다.

검증이 완료되면 Chatbot이 페르소나가 반영된 최종 답변을 생성하며, 마지막으로 Summarize 노드에서 핵심 대화 맥락을 요약해 컨텍스트 길이를 관리한다(토큰 절약 및 장기 대화 안정화).

AGENT	용도	사용 API
WEB SEARCH	운영 시간과 같은 최신 정보 검색	Google Search Gounding, 실제 정보를 기반으로 Gemini 2.5 라이트 모델 응답
RAG	숙박, 맛집, 카페, 레저활동 정보 검색	Chroma DB의 메타데이터에 ID를 저장하여, RAG 검색을 여러 번 수행하더라도 중복 결과가 발생하지 않도록 처리

WEATHER	부산 날씨 검색	Open Weather API 사용, 부산의 향후 5일간의 날씨 예보 검색
---------	----------	---

4.4.3 최신성 문제 해결(Web Search)

LLM의 학습 컷오프(2025년 1월)와 Vector DB의 시의성 한계를 극복하기 위해 **FreshQA 메커니즘**을 도입하였다. **질문 유형을 자동 분류**하고, 최신 정보가 필요한 경우에만 외부 검색을 수행하여 비용과 속도 최적화를 하였다.

사용자 질문을 분석하여 아래와 같이 총 4가지 유형으로 분류하였고, 각 유형에 맞는 처리 전략을 결정하였다.

- 4가지 유형

구분	설명
Type A	변하지 않는 역사적 사실, 기본 상식, 정의 또는 주어진 정보로 계산 가능한 질문
Type B	최신성이 요구되거나 시간에 따라 변동 가능한 정보로, 외부 검색이 필수인 질문
Type C	질문의 전제 자체가 성립하지 않는 경우 (존재하지 않는 대상, 시간적 모순, 사실 왜곡 등)
Type D	미래의 불확실한 예측을 요구하는 질문

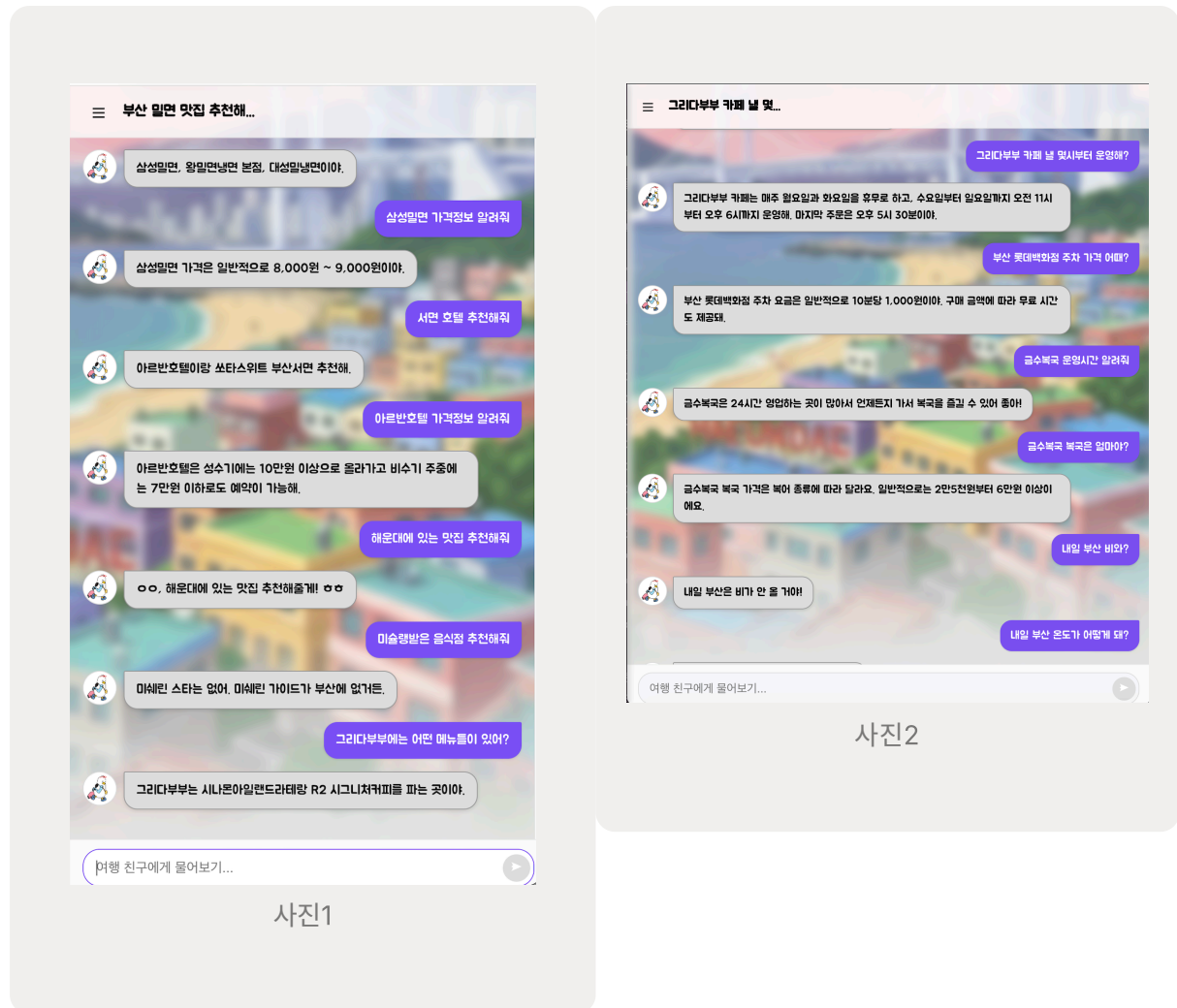
최신 정보(Type B)로 분류된 질문은 아래와 같이 3단계 프로세스를 거쳐서 정보를 추출하였다.

1. Keyword 추출: 질문에서 검색에 적합한 핵심 키워드 추출
2. Wikipedia 검색: 추출된 키워드로 Wikipedia를 검색하여 최신 정보 검색
3. 웹 검색: Wikipedia 검색이 충분하지 않다면, google 웹 검색으로 정보 추출

5. 자체 평가 의견

5.1 전체 프로젝트에 대한 평가

5.1.1 최종 출력 화면



[사진 1]에 따르면 딱딱한 답변 대신 SNS 말투와 반말을 사용해 사용자에게 더 친근하게 다가가고, 핵심만 짧고 굵은 답변을 생성하는 것을 볼 수 있다. [사진 2]에 따르면 RAG(검색 증강 생성)와 실시간 웹 서치를 결합해, 변동 폭이 큰 음식 가격과 영업 시간 및 날씨 정보를 오차 없이 최신 정보로 제공하는 모습을 볼 수 있다.

5.1.2 정량 평가

LLM-as-a-Judge 정량 평가

평가 지표 (Metric)	Base Mode	SFT	DPO	추이 (Base → DPO)
Contextual Relevance (문맥 관련성)	67.55	58.16	33.47	 급격히 하락
Conciseness (간결성)	62.65	60.41	74.69	 상승

Solar Pro2 모델을 활용하여 Base, SFT, DPO 모델의 답변 품질을 비교 평가하였다. SNS 말투와 반말 등 짧은 답변을 유도하는 방향으로 학습이 이루어지면서 간결성 (Conciseness) 지표는 전반적으로 상승하였다. 반면, 간결함(Conciseness)을 목표로 튜닝하는 과정에서 일부 할루시네이션이 발생하여 문맥 관련성(Contextual Relevance) 지표에서는 점수 하락이 발생하였다.

평균 답변 응답 길이

구분	Base Model	DPO Model	개선율
평균 토큰 수	약 125 Tokens	약 20 Tokens	57.3% 절감
생성 속도	느림	매우 빠름	Latency 개선

5.2 한계점 및 아쉬운점

5.2.1 모델 응답 품질의 저하

DPO를 통해 간결함을 극대화하는 과정에서, 복잡한 여행 정보(교통편, 영업시간 등)까지 지나치게 요약하여 정보의 구체성이 일부 손실되는 현상이 확인되었다.

해당 현상의 원인으로 먼저 SFT 데이터가 일상 대화 위주로 구성되어 있어 정보 제공형 대화가 상대적으로 부족했던 점이 있다. 또한 DPO 학습 과정에서 간결한 응답에 대한 선호가 과도하게 반영되면서 정보 축약이 심화된 영향도 있었다. 더불어 RAG 검색 결과를 요약하는 과정에서 핵심 정보가 일부 누락될 가능성 역시 존재한다.

5.3.2 신뢰성 있는 최신 정보 제공의 한계

학습 및 DB 구축 이후 발생하는 폐업, 휴무, 가격 변동, 신규 매장과 같은 변화는 수동 수집 기반인 현재의 RAG 구조만으로 즉각적인 반영이 어렵다. Vector DB가 시간 경과에 따라 노후화되면서 사용자에게 제공되는 정보의 정확성이 저하될 수 있다.

또한 Web Search의 경우, 쿼리 생성 단계에서 품질이 흔들리거나 검색 결과의 신뢰도가 낮을 경우 전체 응답 정확도가 함께 저하될 수 있다. 이러한 한계로 인해 정보 출처와 시점에 따라 답변의 신뢰도 편차가 발생할 가능성이 존재한다.

5.3 향후 개선 방향

5.3.1 DPO 학습 기준 재정의

현재 DPO 학습이 간결함에 과도하게 편향되어 있어, 정보 완전성을 새로운 평가 기준으로 재정의할 필요가 있다. 구체적으로 다음과 같은 선호쌍 구성을 통해 DPO를 재학습할 수 있다:

- **Rejected:** 짧지만 핵심 정보(교통편, 영업시간, 가격 등)가 누락된 답변
- **Chosen:** 친근한 말투를 유지하면서도 필요한 정보를 완전하게 제공하는 답변

이를 통해 친근함과 정보 완전성 사이의 균형을 맞추고, 사용자에게 실질적으로 유용한 답변 생성 능력을 강화할 수 있다.

5.3.2 자동 업데이트 파이프라인 구축(MLOps)

현재 RAG 시스템은 수동 수집 기반으로 구축되어 시간이 지남에 따라 정보가 노후화되는 한계가 있다. 이를 해결하기 위해 Airflow 등의 워크플로 오케스트레이션 도구를 활용한 자동화 파이프라인 구축이 필요하다.

주요 구성 요소

- **데이터 수집:** 월 단위로 카카오맵, 비짓부산 등 주요 소스로부터 최신 여행 정보 크롤링
- **DB 갱신:** 수집된 데이터의 전처리, 청크 분할, 중복 제거 후 Vector DB 자동 업데이트
- **모니터링:** 파이프라인 실행 성공/실패 알림 및 DB 크기, 검색 성능 추적

이를 통해 사용자에게 항상 최신 정보를 제공할 수 있는 지속 가능한 RAG 시스템을 구축할 수 있다.

6. 개인회고

김영현

- 나는 내 학습목표 달성을 위해 무엇을 어떻게 했는가?
 - RAG 구현, LangGraph refactoring
- 전과 비교해서, 내가 새롭게 시도한 변화는 무엇이고, 어떤 효과가 있었는가?
 - 초반에 팀원분들과 코드 리뷰를 진행하였고 혼자서 코드를 구현 할 때는 발견할 수 없었던 에러 취약 코드 등을 발견할 수 있었다.
- 나는 어떤 방식으로 모델을 개선했는가?
 - RAG 구현을 위한 카카오맵 크롤링과 리트리버를 랭체인 패키지를 이용하여 구현하였다.
 - LangGraph 구조를 잡고 리팩토링 하였다.
- 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?
 - 코드 구현에만 머무르지 않고 구현 대상에 대한 정확한 개념을 이해하는 것이 중요하다는 점을 깨달았다. 결국 에러를 수정하거나 성능을 개선하는 과정에서도 핵심은 구현 대상에 대한 명확한 개념 이해에 있다는 것을 알게 되었다.
- 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?
 - 패키지를 통해 구현을 하다보니 구현 자체는 쉬웠지만 동작 방식 등 중요한 부분을 완벽하게 이해하지 못한 것 같다.
 - 크롤링 작업 직 후 멘토링을 통해 크롤링 자동화를 피드백 받았지만 시간과 능력의 한계로 구현하지 못했다.
 - 담당 분야 이외의 기술 스택에 대해서는 학습이 충분하지 못했다.
- 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇인가?
 - 외부 패키지를 이용해 구현하더라도 결과만 사용하는데 머무르기보다 내부 코드 구조를 살펴보거나 공식 문서를 확인하며 작동 방식과 핵심 개념을 이해하려는 시도를 해보고자 한다.

윤준상

- 나는 내 학습목표 달성을 위해 무엇을 어떻게 했는가?
 - 이전에 수행했던 프로젝트에 대한 코드 리뷰를 다시 복습하였다.
 - 해당 프로젝트와 관련하여 베이스라인 코드를 짜보는 연습을 하였고, 파이프라인과 관련하여 프로젝트 방향성에 관해 다양한 자료를 수집하며 고민하는 시간을 가졌다.

- 전과 비교해서, 내가 새롭게 시도한 변화는 무엇이고, 어떤 효과가 있었는가?
 - 처음에는 Streamlit을 통한 구현으로 session state를 활용하여 채팅방별 대화 내용이 섞이지 않도록 세션 독립성을 확보하였다. 이후 Streamlit 특유의 전체 스크립트 재실행이라는 한계점을 확인, 이를 극복하고 구현을 정교화하기 위해 React로 전환하는 작업을 진행하였고, 이를 통해 프론트엔드와 백엔드를 분리하는 방향으로 가닥을 잡았다. 이외에도 디자인적인 요소로 반응형 사이드바나 AI 응답 대기 시간을 고려한 스피너, 실시간 타이핑 효과 등을 넣었다.
- 나는 어떤 방식으로 모델을 개선했는가?
 - 페르소나가 유지되는데 있어서의 방향성과 방법론(프롬프팅, DPO, 학습용 여행질문 생성 등)에 관한 고민을 하였고 이에 대한 대안들을 지속적으로 제시하였다.
- 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?
 - 프론트엔드 작업을 해보면서 구현을 어떻게 할지와 프로젝트 진행 상황에 맞춘 개발 진행을 해본 점이 만족스러웠다.
- 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?
 - 중간에 프론트엔드로 넘어간 이후 구현 로직에 관한 내용을 구체적으로는 완전히 이해하지 못한 점이 아쉬웠다.
 - 중간에 노트북 고장 이슈가 있어 3~4일 공백 기간이 생겼던 점이 아쉽다.
- 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇인가?
 - 논문을 직접 읽어보면서 방법론에 관한 서칭을 하는 것
 - 프로젝트 전체 코드를 하나하나 다 분석해보고 복습하는 것

장세현

- 나는 내 학습목표 달성을 위해 무엇을 어떻게 했는가?
 - 친구형 말투를 **학습**으로 안정적으로 만들기
 - 실패 원인을 데이터/학습설계 관점에서 진단하는 능력
 - 에이전트 워크플로우 설계
- 전과 비교해서, 내가 새롭게 시도한 변화는 무엇이고, 어떤 효과가 있었는가?

이전 프로젝트에서 DPO 학습이 기대만큼 동작하지 않았던 경험이 있어, 이번에는 DPO를 학습방법의 문제가 아니라 **데이터/선호쌍 품질의 문제**로 보고 재설계했다.

또한 LangGraph를 도입해 Tool을 단순 호출이 아니라 상태 기반 워크플로우로 구조화했고, 결과적으로 기능 확장(툴 추가/라우팅 변경)이 쉬워졌으며, 에이전트 동작을 모듈 단

위로 분리해 디버깅이 수월해졌다.

추가적으로, RAG 답변의 근거성/정확성과 페르소나 응답의 일관성을 빠르게 검증하기 위해 LLM-as-a-Judge 평가를 시도했다. 이를 통해 사람이 모두 확인하기 어려운 반복 실험에서 여러 유형을 분류하고, DPO 선호쌍 품질 및 RAG 응답의 근거 포함 여부를 점검하는 자동 평가 루프를 설계하려 했다.

- 나는 어떤 방식으로 모델을 개선했는가?

초기에는 SNS 대화 데이터를 싱글턴 형태로 학습에 사용했는데, 생성 답변이 지나치게 짧고 문맥을 반영하지 못했다. 나는 데이터 구성이 단순해 모델이 표면 패턴에 과적합했기 때문이라고 해석했다. 이후 멀티턴 데이터로 전환했지만, 여행 정보 질문에도 불구하고 모델이 반문을 반복하는 문제가 발생했다. 이는 멀티턴 안에 정보 제공이 아닌 잡담/탐색/관계 유지 발화가 섞여 있어, 모델이 “정보 제공”보다 “대화를 이어가기” 전략을 학습했기 때문이라고 판단했다.

이에 따라 데이터 전처리를 다시 수행해 여행 주제 대화 중에서도 ‘정보 요청 → 정보 제공’ 구간을 중심으로 추출하고, 답변이 나오기 위해 필요한 앞 문맥까지 함께 포함해 학습 데이터를 재구성했다. 또한 QLoRA 설정은 학습 안정성을 위해 보수적으로 조정했다.

- 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?

데이터를 단순히 “싱글턴 → 멀티턴”으로 바꾸는 수준이 아니라, 대화의 목적(정보 요청 → 정보 제공)을 기준으로 학습 데이터를 재구성한 이후부터 응답 품질이 눈에 띄게 안정화되었다. 여행 정보 질문에서 불필요한 반문이 줄고, 질문 의도에 맞는 정보(장소/동선/예산/일정)를 직접 제시하는 형태로 답변 전략이 정렬되는 변화를 확인했다. 또한 LangGraph 기반으로 Tool 흐름을 상태 기반으로 구조화하면서, 기능 추가와 라우팅 변경이 쉬워져 반복 실험과 디버깅 속도가 개선되었다.

이 과정에서 얻은 가장 큰 깨달음은, 친구형 페르소나는 단순히 “친근한 말투”만 학습시키는 문제가 아니라 어떤 상황에서 정보를 제공하고, 어떤 상황에서 되묻고, 어떻게 계획을 확정하는지에 대한 ‘대화 전략’ 전체가 학습되는 문제라는 점이다. 따라서 모델 개선의 핵심은 학습 기법을 바꾸는 것보다 먼저 서비스 목표에 맞는 데이터 구조(입력 문맥 + 목표 응답)를 설계하고 평가 기준을 고정하는 것임을 체감했다.

- 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

학습 과정에서 관찰되는 training/eval loss가 실제 사용자 품질을 충분히 설명해주지 못했다. Loss가 안정적으로 감소하더라도, 실제 대화에서는 페르소나 일관성(말투/대화 전략)이나 정보 제공의 적절성, 그리고 RAG 기반 답변의 근거성 같은 “서비스 품질”이 그대로 따라오지 않는 경우가 있었다. 이전 프로젝트에서 이 문제를 인지해 LLM-as-a-Judge 평가를 도입했지만, Judge 역시 기준의 흔들림/편향 가능성이 있어 단독 지표로 신뢰하기 어려웠고, 결국 품질을 확정할 평가 기준을 충분히 마련하지 못한 점이 아쉬웠다.

또한 신뢰 가능한 답변을 위해 RAG 데이터를 구축했지만, 두 가지 현실적인 한계가 있었다. 첫째, MLOps 관점에서 주기적인 데이터 업데이트/재색인 파이프라인을 구축하지 못해 최신성이 시간이 지날수록 저하될 수 있다. 둘째, RAG가 **신뢰 가능하다**는 것은 단순히 문서를 붙였다는 의미가 아니라, 답변이 실제로 검색된 근거에 의해 뒷받침되는 와 검색 자체가 적절했는지를 검증해야 하는데, 이를 정량적으로 측정·모니터링하는 체계가 부족했다.

- 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇인가?

이번 프로젝트의 핵심 교훈은 모델의 추가 학습(SFT/DPO)에 앞서, 무엇이 '정답인'에 대한 정성적/정량적 지표(Rubric)를 정의하고 이를 재현 가능하게 운영하는 것이 우선이라는 점이었습니다.

다음 프로젝트에서는 기획 단계부터 페르소나 유지력, 근거성, 최신성에 대한 상세 가이드를 문서화하고, 이를 검증할 평가 파이프라인을 선제적으로 구축하겠습니다. 또한, RAG를 단순한 모듈이 아닌 지속 가능한 시스템으로 정의하여, 데이터 업데이트 및 재색인 자동화를 포함한 MLOps 설계에 집중함으로써 서비스의 신뢰성을 지속적으로 확보하는 방향으로 진행하겠습니다.

주현민

- 나는 내 학습목표 달성을 위해 무엇을 어떻게 했는가?
 - 프로젝트 발표 자료 제작, 발표 역할을 수행하며, 프로젝트의 전체 코드, 방법론들을 공부했다.
- 전과 비교해서, 내가 새롭게 시도한 변화는 무엇이고, 어떤 효과가 있었는가?
 - 챗봇 모델 SFT 베이스라인 코드를 작성했다.
- 나는 어떤 방식으로 모델을 개선했는가?
 - 싱글턴 QA 데이터를 사용한 SFT파이프라인을 만들었다.
 - 눈에 띄는 개선이 없었지만, SFT를 위해서 데이터를 어떤 형태로 가공하고, 학습시킬지 고민하는 계기를 제공했다.
- 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?
 - SFT파이프라인을 구축한 결과 실험 관리가 간편해졌다.
 - 이전 프로젝트에서는 QA 쌍의 데이터를 학습했지만, 이번 프로젝트는 정해진 데이터 구조가 없다.
무엇을 위한 학습인지 생각하고 데이터 입력, 모델링을 하는 게 중요함을 제대로 깨달았다.

- 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?
 - 어떤 형태의 데이터로 SFT할지 고민을 제공했지만, 성능 개선이 없었던 이유가 데이터 품질 문제인지, 데이터 가공 과정의 문제인지 답을 찾지 못했다.
 - 친구처럼 짧은 문장 안에 필요한 정보를 제공하도록 모델의 짧은 답변 생성을 목적으로 학습하였는데, 필요한 정보를 누락한 짧은 응답을
- 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇인가?
 - top-k 단어, top-k n-gram은 무엇인지 등, 데이터의 품질을 검증 EDA를 더 꼼꼼하게 할 것이다,
 - 방법론 리서치를 LLM 위주가 아닌 깃허브, 논문 위주로 시도해보겠다.

한지석

- 나는 내 학습목표 달성을 위해 무엇을 어떻게 했는가?

RAG 시스템 구축 중, 사람이 직접 작성한 정답 데이터(GT)를 확보하기 어렵다는 페인 포인트를 발견했다. GT 없이도 LLM을 활용해 평가가 가능한 RAGAS(RAG Assessment) 프레임 워크를 학습하고, RAG Triad 기반 평가 원리를 파악했다. 검색 성능의 핵심인 Context Recall과 Context Precision을 중 성능 지표로 정의했다. Ragas 라이브러리를 활용하여 테스트 데이터셋 생성 및 점수 산출까지의 파이프라인을 구축하였다.
- 전과 비교해서, 내가 새롭게 시도한 변화는 무엇이고, 어떤 효과가 있었는가?

기존의 단순 데이터 적재 방식에서 벗어나, **LLM을 활용한 PDF 텍스트 정제 전처리**를 도입하였다. 이를 통해 문서 내 노이즈를 제거함으로써 RAG 시스템의 답변 정확도와 맥 이해도를 대폭 개선하였다.

기존의 수동 방식에서 벗어나 **distilabel과 ragas를 활용한 데이터 생성 및 평가 파이프라인**을 새롭게 구축했다. 이를 통해 고품질의 합성 데이터를 빠르게 확보하고, 정량적인 지표로 RAG, 챗봇을 검증하는 등 개발 사이클의 효율을 극대화했다.
- 나는 어떤 방식으로 모델을 개선했는가?

모델이 생성하는 답변의 정확도를 높이기 위해 **실제 여행 가이드북 데이터를 기반으로 RAG 시스템을 구축**했다. 이를 통해 사용자에게 정확하고 전문적인 맞춤형 여행 정보를 제공하도록 모델을 개선했다.
- 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?

단순한 모델 구현을 넘어 **학습·평가 체계를 완성**하는 성과를 거두었으며, 특히 RAG나 DPO와 같은 복잡한 구조일수록 기술적 원리에 기반한 **철저한 사전 검증**이 결과의 신뢰

도를 결정짓는 핵심임을 학습했다.

- 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

DPO 학습 과정에서 특정 페르소나(말투)의 반영과 모델의 어휘력 보존 사이의 **트레이드오프(Trade-off)**를 극복할 수 있는 **최적의 지점을 찾지 못해**, 두 가지 성능을 동시에 확보하는 데 한계가 있었다.

평가 프롬프트와 지표를 LLM 생성에 의존하다 보니, **판단 기준의 독립성과 신뢰도를 완전히 확보**하는 데 어려움이 있었다. 특히 가이드북 기반 RAG의 특성상 정교한 검증이 중요함에도 불구하고, **Human-in-the-loop를 통한 상관관계 측정**까지 나아가지 못한 점이 가장 아쉽다.

- 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇인가?

기술 도입 시 철저한 사전 리서치를 통해 작동 원리에 대해 설명가능할 정도로 더 깊게 이해하기

코드 작성 시 LLM 의존도를 낮춰 **자체적인 기술적 문제 해결 역량**을 강화